Introduction to Simulink

Basic Blocks Used in Control Systems

1. Introduction

Simulink is a graphical simulation environment integrated with MATLAB, widely used for modeling, simulation, and analysis of dynamic systems. Unlike traditional coding approaches, Simulink allows users to represent systems using block diagrams, enabling intuitive visualization of signal flow and system behavior.

This report presents the modeling and simulation of a basic control system using fundamental Simulink blocks. The objective is to understand system representation, signal flow, and response analysis using a graphical approach.

2. System Modelling

A basic control system model was developed using a combination of source, processing, and output blocks. The system consists of:

- A **Step input** representing a sudden change in input
- A **Transfer Function block** representing system dynamics
- A **Scope block** for output visualization

This structure demonstrates the fundamental concept of input → system → output.

3. Apparatus / Software Required

For this experiment, no physical hardware was required. Everything was done on a computer with the following software:

- A standard computer system with adequate processing capability
- MATLAB software — the primary computational environment
- Simulink — the graphical simulation toolbox within MATLAB

4. Theory

4.1 What is Simulink, and Why Does It Exist?

When I first heard about Simulink, I thought it was just another way to write MATLAB code. But it's actually quite different — and honestly, more intuitive in many ways. Simulink is a visual, block-diagram based simulation environment. Instead of writing equations line by line in a script, you represent your system using interconnected blocks, where each block handles a specific operation.

Think of it like building with LEGO pieces, but each piece has a mathematical function. You snap them together, and when you hit "run", the entire system comes to life. This visual approach

is one of the reasons Simulink has become so popular — you can see the flow of signals through a system in real time, without needing to mentally trace through hundreds of lines of code.

4.2 Why Simulink Matters Specifically for Control Systems

Control systems are inherently complex. They deal with feedback loops, differential equations, system responses, and stability analysis — and doing all of this manually through mathematical derivation is time-consuming and error-prone. Simulink steps in as a practical solution to this problem.

With Simulink, you can model a control system — say, a PID controller with a plant transfer function — in just a few minutes by connecting the right blocks. Then, by hitting the simulation button, you can instantly observe how the system behaves over time. If the response isn't what you expected, you just tweak a parameter and run it again. This kind of rapid iteration would take far longer if done on paper or purely through code.

In short, Simulink simplifies the modeling process, speeds up testing, and gives you clear graphical feedback about how your system is performing — all of which are hugely valuable when working with real-world control applications.

4.3 Basic Components of a Simulink Model

Every Simulink model, no matter how simple or complex, is made up of three fundamental elements: blocks, signal lines, and input/output signals. Understanding these three components is the foundation of working with Simulink.

Blocks

Blocks are the building units of any Simulink model. Each block represents a specific mathematical function or system operation. For example, a Gain block multiplies whatever signal it receives by a constant value, while a Transfer Function block represents the mathematical relationship between the input and output of a dynamic system. There are blocks for virtually every kind of operation you might need — addition, integration, signal generation, display, and more.

Signal Lines

Lines connect the blocks together and represent the flow of data — or signals — from one block to another. When you draw a line from the output of one block to the input of another, you're essentially telling Simulink: "take what this block produces and pass it along to the next block." This is what gives Simulink its intuitive, visual feel.

Input and Output Signals

These define what enters the system (typically from source blocks) and what comes out (captured by sink blocks). In a typical control system model, a step signal might be the input, and a Scope showing the system's time response would be the output.

4.4 Commonly Used Block Categories in Control Systems

Simulink's library contains hundreds of blocks, but for control system work, a handful of categories come up repeatedly. Here is a closer look at each one.

4.4.1 Source Blocks — Where Signals Come From

Source blocks are how you feed a signal into your model. They don't receive any input — they generate it. The most commonly used source block in control systems is the Step block. It produces a signal that stays at zero until a specified time, then jumps to a set value and stays

there. This kind of input is widely used because it lets you test how a system responds to a sudden change — which is exactly what happens in real-world scenarios like switching on a motor or changing a reference setpoint.

Other source blocks include the Sine Wave block (useful for testing frequency response) and the Constant block (provides a steady, unchanging value). For most basic control experiments, though, the Step block is the go-to choice.

4.4.2 Sink Blocks — Where Signals End Up

If source blocks are the input side of the model, sink blocks are the output side. The Scope block is by far the most important one here. It works like an oscilloscope — it receives the signal coming out of your system and displays it as a time-domain graph. When you want to see how your system responded to the step input — did it settle smoothly? Did it overshoot? How fast did it stabilize? — you look at the Scope output.

The Display block is another sink option — it shows a numerical value rather than a waveform. For observing dynamic system behavior over time, though, the Scope is much more informative.

4.4.3 Continuous Blocks — Representing Dynamic Systems

These are at the heart of control system modeling. The Transfer Function block is the most critical one in this category. It lets you define a linear dynamic system using its transfer function — essentially a ratio of two polynomials representing the output over the input in the frequency domain (Laplace domain). You simply enter the numerator and denominator coefficients, and Simulink handles the mathematics.

For example, a simple first-order system with transfer function 1/(s+2) can be entered by setting the numerator to [1] and the denominator to [1, 2]. Simulink then simulates how that system responds to whatever input you connect to it.

The Integrator block is also part of this category. It performs the mathematical operation of integration on the incoming signal — essentially computing the area under the curve over time. This is particularly useful when modeling systems described by differential equations.

4.4.4 Math Operation Blocks — Doing the Arithmetic

Control systems involve a lot of mathematical operations happening in parallel, and Simulink provides dedicated blocks for each. The Gain block simply multiplies the incoming signal by a constant. You set that constant in the block's parameter, and it applies it to every sample of the signal as the simulation runs.

The Sum block is perhaps even more important — it adds or subtracts signals together. This is what makes feedback loops possible. In a standard closed-loop control system, you subtract the actual output from the desired reference to get the error signal, and the Sum block is what performs that subtraction. You can configure each input of the Sum block to be either a positive (addition) or negative (subtraction) input.

4.4.5 Signal Routing Blocks — Managing Multiple Signals

When your model grows more complex and you have multiple signals flowing through it, signal routing blocks help you manage them cleanly. The Mux (multiplexer) block combines several separate signals into a single bundled signal — this is useful when you want to display multiple signals on one Scope simultaneously. The Demux (demultiplexer) does the reverse — it splits a combined signal back into its individual components.

These blocks become increasingly important as models scale up. Even in simple experiments, using a Mux to overlay multiple waveforms on a single Scope can make comparisons much easier to understand.

4.5 How Simulink Actually Works — Step by Step

The workflow in Simulink follows a clear sequence. First, you open MATLAB and launch the Simulink environment either through the toolbar or by typing simulink in the command window. From there, you open the Simulink Library Browser — a catalogue of all available blocks organized by category.

You create a new blank model, which opens as an empty canvas. Then you drag the blocks you need from the library and drop them onto this canvas. After arranging them logically, you draw connections between them by clicking and dragging from the output port of one block to the input port of another. Once everything is connected, you double-click each block to set its parameters — entering the transfer function coefficients, the gain value, the step time, and so on.

When everything is in place, you hit the Run button. Simulink solves the mathematical equations behind the scenes in real time, and at the end of the simulation, you open the Scope to view the output waveform. The whole process is fast, visual, and — once you get the hang of it — quite satisfying to use.

4.6 Advantages of Using Simulink

Having worked through this experiment, it became clear why Simulink is so widely adopted, not just in academia but in industry too. The most obvious advantage is visual clarity — you can look at a Simulink model and immediately understand the structure of the system. There is no ambiguity about which block feeds into which.

It is also quick to use. Building a basic closed-loop system that would take considerable effort to code from scratch can be assembled in Simulink within minutes. Simulation runs fast, and the graphical output from the Scope makes interpreting results straightforward.

Perhaps most importantly, Simulink scales well. The same concepts that apply to simple first-order models carry over directly to complex, multi-loop systems. So once you understand the basics — which this experiment was designed to establish — you have a solid foundation for tackling more advanced simulations.

5. Important Blocks — A Closer Look

Before diving into the actual procedure, it helps to understand the specific blocks used in this experiment and what each one actually does in practice.

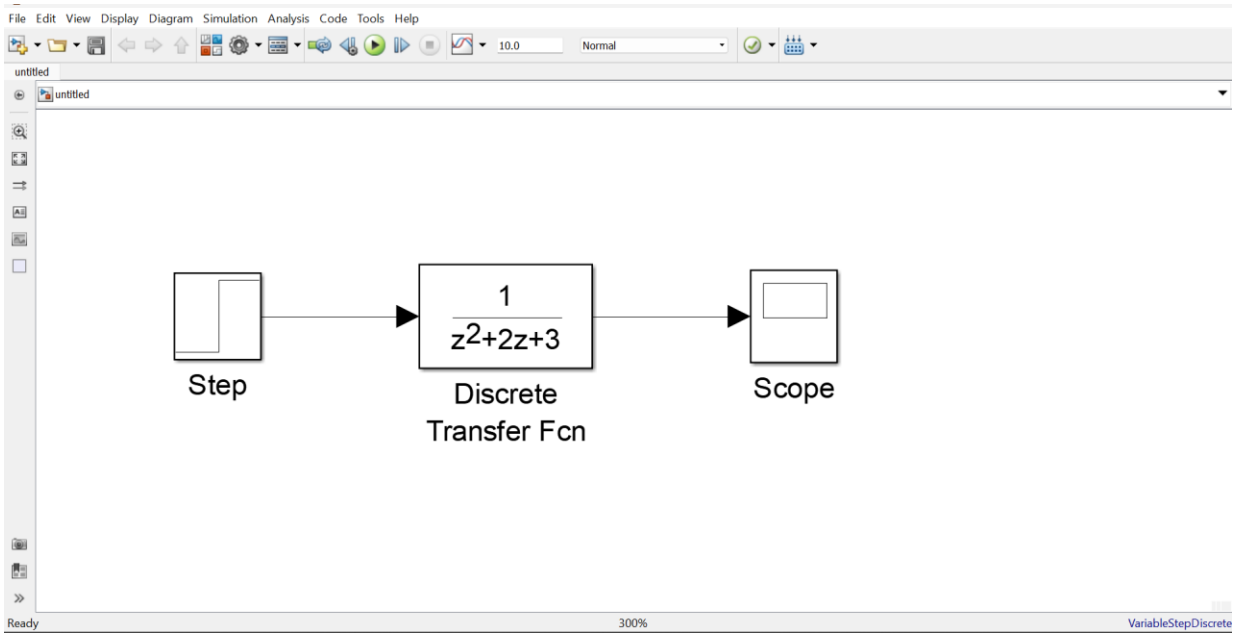
Block Name	Role in the Model
Step Block	Generates the input signal — a sudden jump from 0 to 1 at a defined time. This simulates a real-world command like 'switch on' or 'change reference'.
Transfer Function	Represents the mathematical model of the system being controlled. You enter the numerator and denominator polynomials to define system dynamics.
Gain Block	Scales a signal by a fixed constant. Used to model amplifiers, proportional controllers, or any system component with a constant scaling factor.

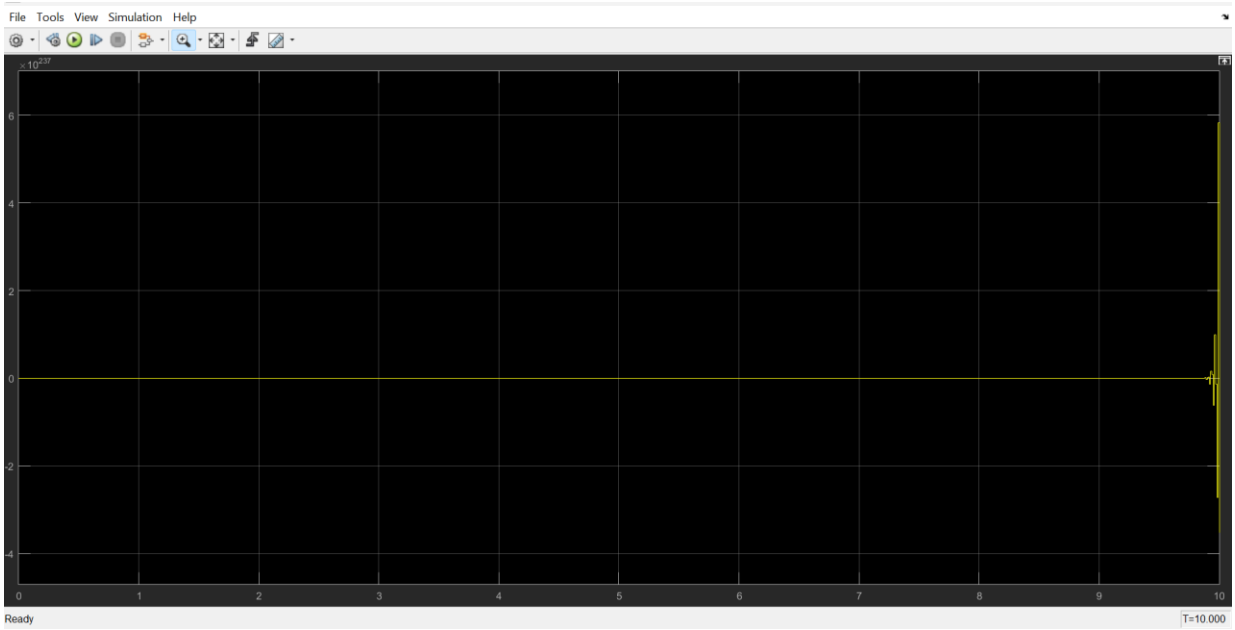
Sum Block	Adds or subtracts multiple incoming signals. Essential for feedback loops — subtracts actual output from desired input to calculate the error.
Scope Block	Displays the signal waveform over time. Acts as an oscilloscope to visually observe the system response after the simulation runs.
Integrator Block	Integrates the incoming signal with respect to time. Represents the mathematical integration operation and appears in models described by differential equations.

6. Implementation

The steps followed during this experiment were straightforward, though they require care at each stage to ensure the model behaves correctly. Here is how the process unfolded:

1. Open MATLAB software on the computer system.
2. In the MATLAB command window, type simulink and press Enter. This launches the Simulink start page.
3. Open the Simulink Library Browser — this is the catalogue of all available blocks.
4. Create a new blank model from the File menu or the start page.
5. From the Sources library, drag the Step block onto the model canvas. This will serve as the input signal.
6. From the Continuous library, drag the Transfer Function block and place it to the right of the Step block.
7. From the Sinks library, drag the Scope block and place it to the right of the Transfer Function block.
8. Connect the output port of the Step block to the input port of the Transfer Function block by clicking and dragging a signal line between them.
9. Connect the output port of the Transfer Function block to the input of the Scope block.
10. Double-click the Transfer Function block and enter the desired numerator and denominator values to define your system.
11. Double-click the Step block and configure the step time, initial value, and final value as needed.
12. Check the simulation stop time in the model settings and set it appropriately.
13. Click the Run button (the green play button in the toolbar) to start the simulation.
14. Once the simulation completes, double-click the Scope block to open the output window and observe the system response.





7. Simulink Model Description

The model built in this experiment is intentionally simple — a three-block chain that demonstrates the fundamental concept of input, processing, and output. The Step block generates a unit step signal, meaning the signal goes from 0 to 1 at t = 1 second and stays there for the rest of the simulation. This input travels along the signal line to the Transfer Function block.

The Transfer Function block receives this step input and processes it according to the system dynamics defined by the numerator and denominator coefficients. Mathematically, it applies the Laplace domain transfer function to the incoming signal and produces a corresponding output. The nature of this output — how quickly it rises, whether it overshoots, how long it takes to settle — depends entirely on what transfer function you've entered.

That processed output then flows into the Scope block, which logs the values at each simulation timestep and displays them as a continuous waveform when opened after the simulation ends.

Even this minimal three-block model communicates a lot about how Simulink works — and how the structure of the model directly reflects the physical or mathematical system it represents.

8. Observations

After running the simulation and examining the Scope output, several interesting observations were made:

Output Waveform Shape

The Scope displayed a smooth, continuous waveform — not a sharp jump but a curve that gradually rose in response to the step input and then leveled off at the final value. This characteristic shape is typical of first-order and second-order systems and is exactly what control system theory predicts for a step response.

Effect of the Transfer Function

Changing the values in the transfer function's numerator and denominator visibly altered the shape of the response. A system with a smaller time constant settled faster, while one with a larger time constant took longer. This demonstrated that the transfer function is truly the mathematical description of the system — it directly controls the output behavior.

Effect of the Gain Block

When a Gain block was inserted into the model and its value adjusted, the output amplitude changed proportionally. Increasing the gain made the output reach a higher steady-state value, while reducing it brought the output down. This confirmed that the Gain block acts as a simple scalar multiplier on the signal passing through it.

Step Input Behavior

The step input produced a characteristic curve — a smooth exponential-like rise that is commonly referred to as the step response in control theory. The system did not respond instantaneously; instead, it built up gradually, which is realistic and expected for most physical systems.

9. Precautions

A few important things to keep in mind when working with Simulink, especially for beginners:

- Always verify that all blocks are properly connected — a missing or broken connection will cause the simulation to throw an error or produce incorrect results. Simulink usually highlights broken connections with a red dashed line.
- Enter the transfer function coefficients carefully. A small typo in the numerator or denominator can completely change the system's behavior in ways that are hard to diagnose.
- Check the simulation stop time before running. If it's set too short, you may not see the full response; if it's too long, the simulation may take an unnecessarily long time to complete.
- Use appropriate block parameters for the context. For example, the step time in the Step block should be set so that the step occurs within the simulation window.
- Make sure the MATLAB and Simulink versions are compatible with the features you're using, especially if working with older or newer versions of the software.

10. Result

The experiment was completed successfully. A basic Simulink model was built using the Step block as input, a Transfer Function block to define the system dynamics, and a Scope block to capture the output. The simulation ran without errors, and the output waveform was clearly visible on the Scope, showing the expected step response behavior of the system. The effect of modifying block parameters — such as the transfer function coefficients and the gain value — was also observed and confirmed to match theoretical expectations.

11. Conclusion

This experiment gave me a hands-on introduction to Simulink that went beyond theory. What became clear quite quickly is that Simulink genuinely makes control system modeling accessible. The visual nature of the tool — where signals flow visibly from one block to the next — makes it easy to understand what's happening at each stage of the system, something that raw code doesn't always communicate as clearly.

The blocks studied here — Step, Transfer Function, Gain, Sum, Scope, and Integrator — form the backbone of most basic control system models. Understanding what each one does, and how they connect together to form a working simulation, is an essential first step. Everything more complex in control system design builds directly on this foundation.

Most importantly, this experiment reinforced the idea that simulation is not just a theoretical exercise — it is a practical tool for testing ideas before implementing them in real systems. That perspective is central to how engineering is practiced today, whether you're working on control systems for satellites, spacecraft, or ground-based applications.